

的结构。小程序渲染层的运行环境是一个 WebView，而 WebView 无法直接理解 WXML 标签，所以需要经过编译。

微信开发者工具内置了一个二进制的 WXML 编译器，这个编译器接受 WXML 代码文件列表，处理完成之后输出 JavaScript 代码，这段代码是各个页面的结构生成函数。

WXSS（WeiXin Style Sheets）是一套样式语言，用来决定 WXML 的组件应该怎么显示。为了适应广大的前端开发者，WXSS 具有 CSS 的大部分特性。同时为了更适合开发微信小程序，WXSS 对 CSS 进行了扩充以及修改。

微信客户端在运行小程序的逻辑层时只需要加载一个 JS 文件（即 app-service.js），而小程序框架允许开发者将 JavaScript 代码写在不同的文件中，所以在代码上传之前，微信开发者工具会对开发者的 JS 文件做一些预处理，包括 ES6 转 ES5 和代码压缩（开发者可以选择关闭预处理操作），在服务器编译过程将每个 JS 文件的内容分别包裹在 define 域中，再按一定的顺序合并成 app-service.js。其中，对于页面 JS 和 app.js 需要主动 require。

2. 模拟器

小程序模拟器模拟小程序在微信客户端的逻辑和界面表现，方便开发者实时查看代码效果。由于系统差异以及微信客户端特有的一些交互流程，少部分的 API 无法在模拟器上进行模拟，但对于绝大部分的 API 均能够在模拟器上呈现出正确的状态。同时，微信开发者工具提供多种机型尺寸以及自定义机型尺寸功能，方便开发者进行界面样式的机型适配。

18.2.3 网页应用开发环境

移动端网页应用的开发环境包括如下三种：

- (1) HTML5 开发工具，如 DevExtreme。
- (2) 移动设备开发技术。设计在移动设备上运行良好的网站所需的一些主要技术，如跨浏览器兼容性、移动设备设计等。
- (3) 移动端开发技术，如 Native App、Hybrid App 等。

18.3 移动应用架构

应用架构是指按照某种规范和约束将业务能力进行拆分，并由不同应用（系统）承接的结构载体，从而能够实现将拆分后的应用以一种规律有序的方式进行连接并创造业务活动。应用架构分为以下两个不同的层次：企业级的应用架构和系统级的应用架构。

企业级的应用架构起到了统一规划、承上启下的作用，向上承接了企业战略发展方向和业务模式，向下规划和指导企业各个 IT 系统的定位和功能。而系统级的应用架构则是在企业级应用架构的指导下，对单个系统进行设计和实现。应用架构设计可以帮助开发人员更好地理解 and 把握软件系统，从而更好地满足用户需求，提高软件质量，降低开发成本。

移动应用的架构发展历史可以大致分为以下几个阶段。

1. 单体应用阶段

早期的移动应用开发采用单体应用架构，即所有功能模块都集成在一个应用程序中，是一种将所有业务场景的表示层、业务逻辑层和数据访问层放在一个工程中的应用架构模式。例如，开发一个进销存系统，可以将项目打包成 war 包并部署到服务器上，这样的 war 包涵盖了很多模块。

单体应用架构模式的优点是：开发简单、部署简单、运维简单、性能稳定。但是，随着业务的发展，单体应用会变得越来越复杂，代码量也会越来越大，这样会导致开发效率低下、维护成本高。

2. MVC 架构阶段

随着移动应用市场的快速发展，MVC（Model-View-Controller）架构开始逐渐被应用到移动应用开发中。该架构将应用程序分为三层：模型层、视图层和控制器层，实现了代码解耦和代码复用，提高了应用的可扩展性和可维护性。同时，该架构也引入了一些新的问题，如控制器层可能会变得庞大而难以管理。

MVC 架构示意图如图 18-4 所示。下面举一个常见的例子来具体说明 MVC 各自的作用。例如，小时候玩的卡带式游戏机，Control 是主机，一般来说买一个主机就行了，只要不坏，就能一直玩这一类的游戏。View 则是电视机和游戏手柄，电视机可以独立工作，不管输入的是电视信号、影碟机信号还是游戏机信号，它只管显示，而且电视机决定了我们看到的效果是怎样的，如果想要一个尺寸更大的或者彩色的显示效果，只需要买个相应的电视机即可，手柄也是可以换的。Model 则是游戏卡带，它决定了玩的是什么游戏，是魂斗罗还是超级玛丽，而且游戏机主机和电视机生产厂家永远也不知道在上面有可能会运行什么样的游戏。卡带中可能会有游戏代码和存储单元，都根据游戏的需要而设计。

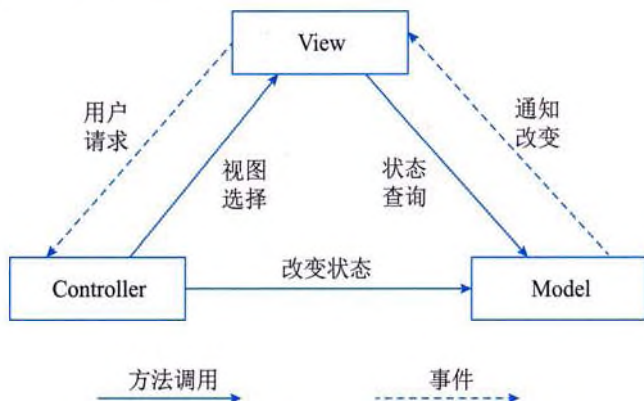


图 18-4 MVC 架构示意图

MVC 主要的框架有 Spring、Struts ZF、.NET。以 SpringMVC 为例来解释 MVC 在 Spring 的工作流程中如何体现的。

SpringMVC 是一个 MVC 的开源框架，SpringMVC=struts2+spring，相当于 Struts2 加上

Spring 的整合。但是这里有一个疑惑，SpringMVC 和 Spring 是什么样的关系呢？有一个很好的解释是：SpringMVC 是 Spring 的一个后续产品，其实就是 Spring 在原有基础上，又提供了 Web 应用的 MVC 模块，可以简单地把 SpringMVC 理解为 Spring 的一个模块（类似 AOP、IOC 这样的模块），我们经常说 SpringMVC 和 spring 无缝集成，其实 SpringMVC 就是 Spring 的一个子模块，所以不需要同 Spring 进行整合。

图 18-5 将前端的视图用 jsp 来代替，DAO 是 MVC 外的另一个层，即数据持久层，主要进行对数据库的操作。

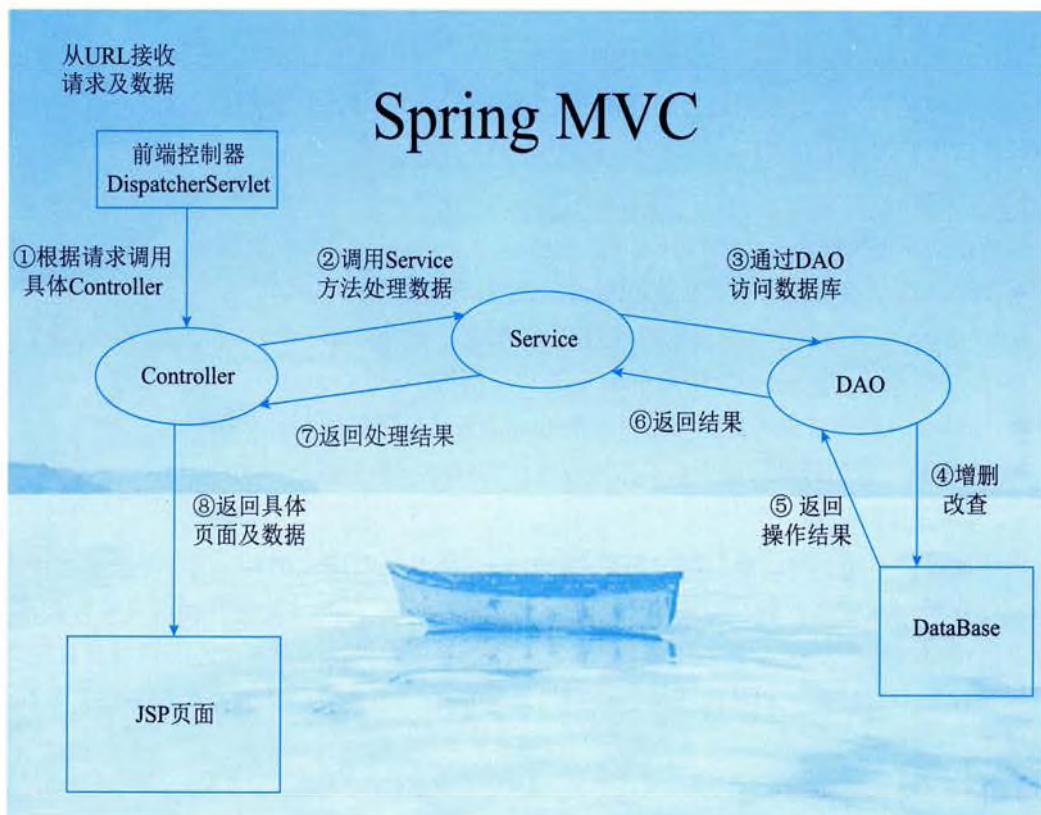


图 18-5 SpringMVC 示意图

SpringMVC 的工作流程如下：

- (1) 用户发起请求到前端控制器（DispatcherServlet）。
- (2) 前端控制器请求处理器映射器（HandlerMapping）去查找处理器（Handle）：通过 xml 配置或者注解进行查找。
- (3) 找到处理器映射器（HandlerMapping）后，向前端控制器返回执行链（HandlerExecutionChain）。
- (4) 前端控制器（DispatcherServlet）调用处理器适配器（HandlerAdapter）去执行处理器（Handler）。
- (5) 处理器适配器去执行 Handler。

- (6) Handler 执行完给处理器适配器返回 ModelAndView。
- (7) 处理器适配器向前端控制器返回 ModelAndView。
- (8) 前端控制器请求视图解析器 (ViewResolver) 去进行视图解析。
- (9) 视图解析器向前端控制器返回 View。
- (10) 前端控制器对视图进行渲染。
- (11) 前端控制器向用户响应结果。

下面是一个 MVC 架构案例。

1) 项目背景

微信是一款由中国腾讯公司开发的即时通信软件，于 2011 年发布。它在移动设备上提供了各种功能，包括聊天、语音通话、朋友圈、支付等。微信是一个非常成功的移动应用，目前在全球范围内拥有数十亿的用户。

2) 技术栈

微信移动应用的技术栈主要包括以下几方面：

- Java。微信移动应用的后端使用Java语言进行开发。
- Objective-C/Swift。微信移动应用的iOS客户端使用Objective-C或Swift进行开发。
- Java/Kotlin。微信移动应用的Android客户端使用Java或Kotlin进行开发。
- Node.js。微信支付和其他一些服务使用Node.js进行开发。
- MySQL。微信移动应用的后端数据存储使用MySQL数据库。

3) 功能模块

微信移动应用的功能模块包括：用户可以在微信中发送和接收文本、图片、语音、视频等消息，并支持一对一和群聊；用户可以添加、删除和管理联系人，并查看联系人的详细信息；用户可以在朋友圈中发布文字、图片、视频等内容，并查看其他用户的动态、点赞和评论；用户可以在微信中进行支付和转账等操作；微信还提供了许多其他的功能，如扫一扫、公众号、小程序等。

4) MVC 架构的具体使用

微信移动应用使用了 MVC 架构，将各个功能模块分为模型层 (Model)、视图层 (View) 和控制器层 (Controller)。以聊天模块为例，具体使用如下：

- 模型层定义聊天消息、聊天记录等数据。在微信中，消息的数据结构包括消息类型、发送者、接收者、内容等。
- 视图层展示聊天界面。在微信中，聊天界面展示了所有聊天消息，包括文本、图片、语音、视频等。
- 控制器层实现消息的发送和接收，聊天记录的存储等逻辑。在微信中，聊天控制器负责发送和接收消息，并将消息存储到本地数据库中。

通过 MVC 架构，微信实现了模块化开发，各个模块之间松散耦合，便于后续扩展和维护。例如，在聊天模块中，如果需要增加一个新的消息类型，只需要修改模型层的数据结构和控制

器层的发送和接收逻辑，而不需要修改视图层的界面展示逻辑，这使得开发更加灵活和高效。

5) 总结

微信移动应用采用了 MVC 架构，将各个功能模块分为模型层、视图层和控制器层。通过 MVC 架构，微信实现了模块化开发，各个模块之间松散耦合，便于后续扩展和维护。微信移动应用的技术栈主要包括 Java、Objective-C/Swift、Java/Kotlin、Node.js 和 MySQL 等。微信移动应用的功能模块包括聊天模块、联系人模块、朋友圈模块、支付模块和其他模块。微信移动应用是一款非常成功的移动应用，为用户提供了丰富的功能和优秀的用户体验。

3. MVP 阶段

在 MVC 的设计中，将 View 和 Controller 的代码放到一起，这样会导致 Activity/Fragment 中的代码越来越多，所以开始分离 VC 中的代码。

将 Activity/Fragment 中复杂的逻辑处理移至另外的一个类（Presenter）中，此时，Activity 其实就是 MVP 模式中的 View，负责 UI 元素的初始化，建立 UI 元素与 Presenter 的关联（Listener 之类），同时自己也会处理一些简单的逻辑，复杂的逻辑交由 Presenter 处理。

模型（Model）层：用于封装与应用程序的业务逻辑相关的数据以及对数据的处理方法。模型层有对数据直接访问的权利，如访问数据库、获取网络数据。Model 不依赖 View 和 Presenter，也就是说，Model 不关心它是如何显示或者如何被操作。

视图（View）层：负责显示数据，与用户进行交互。持有逻辑层的引用。在 Android 中体现为 Activity/Fragment。

逻辑（Presenter）层：负责逻辑处理，通过模型层获取数据，回显到 UI 层，响应用户的行为。

MVP 架构是一种软件设计模式，它将应用程序分为三个层次，即 Model、View 和 Presenter。Model 负责处理数据和业务逻辑，View 负责显示用户界面，Presenter 负责协调 Model 和 View 之间的交互。MVP 架构的目的是降低 View 和 Model 之间的耦合，提高代码的可测试性和可维护性。MVP 架构示意图如图 18-6 所示。

下面是一个 MVP 架构案例。

1) 项目背景

美食家是一款以美食为主题的移动应用程序，该应用程序旨在为用户提供美食推荐、餐厅预订、菜谱分享等服务。在该应用程序中，MVP（Model-View-Presenter）架构被用来实现代码的模块化、可复用性和可测试性。

2) 架构设计

在 MVP 架构中，应用程序被分为三个组件：模型（Model）、视图（View）和逻辑（Presenter）。在美食家应用程序中，这三个组件的具体实现如下：

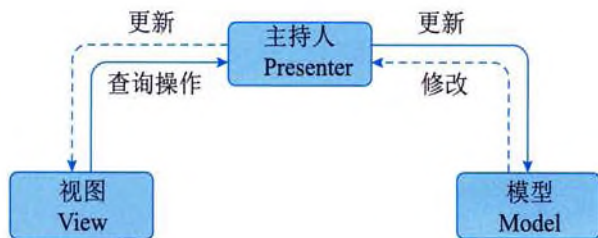


图 18-6 MVP 架构示意图

模型层：美食家应用程序的模型层被设计为一个数据仓库，负责处理数据逻辑和管理数据源。具体来说，该层包含了一个 SQLite 数据库和一个 Retrofit 库。SQLite 数据库被用来存储应用程序的本地数据，如用户信息、菜谱信息等。Retrofit 库被用来处理网络请求和 JSON 数据的解析。

视图层：美食家应用程序的视图层被设计为一个 UI 层，负责展示数据和接收用户的操作。具体来说，该层包含了各种 Android 的 UI 组件，如 TextView、EditText、ListView 等。这些 UI 组件被用来展示美食信息、菜谱信息、用户信息等。

逻辑层：美食家应用程序的逻辑层被设计为一个业务逻辑层，负责处理应用程序的业务逻辑。具体来说，该层通过调用模型层和视图层的 API 来实现业务逻辑。该层使用了 RxJava 和 Dagger2 框架，以实现响应式编程和依赖注入。

3) 开发思路

美食家应用程序的开发思路是将整个应用程序划分为多个功能模块，并将每个模块的代码分别实现在 MVP 架构的三个组件中。在每个组件中，使用接口来定义组件之间的交互，以实现代码的模块化和可复用性。同时，使用依赖注入来管理组件之间的依赖关系，以实现代码的可测试性。

4) 功能模块

美食家应用程序包含以下功能模块：

- (1) 用户注册、登录和个人信息管理。该模块包括用户注册、登录、修改个人信息等功能。
- (2) 美食推荐和搜索。该模块包括美食推荐、搜索美食等功能。
- (3) 餐厅预订和评论。该模块包括餐厅预订、取消预订、餐厅评论等功能。
- (4) 菜谱分享和收藏。该模块包括菜谱分享、收藏菜谱等功能。
- (5) 美食地图和导航。该模块包括美食地图、导航等功能。

以上模块分别被实现在 MVP 架构的三个组件中，以实现代码的模块化、可复用性和可测试性。

5) 总结

美食家应用程序是一款使用了 MVP 架构的移动应用程序，该应用程序采用了模块化、可复用性和可测试性的开发思路，并实现了多个功能模块。同时，该应用程序的架构设计具有良好的可扩展性和可维护性，以满足应用程序的长期需求。

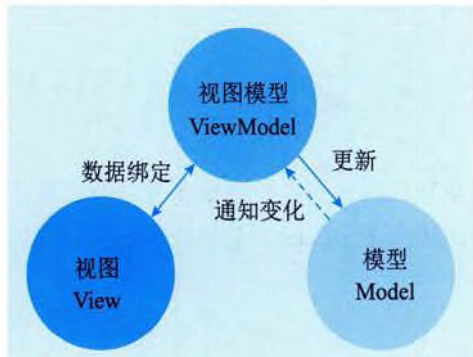


图 18-7 MVVM 架构示意图

4. MVVM 架构阶段

MVVM (Model-View-ViewModel) 架构是在 MVC 架构的基础上发展起来的。MVVM 架构示意图如图 18-7 所示。该架构通过引入 ViewModel 层，进一步解耦了视图和模型，使每个组件的职责更加清晰明确。另外，MVVM 架构还引入了数据绑定和响应式编程的思想，使得应用程序能够更加灵活地

响应用户输入。MVC → MVP → MVVM 这几个软件设计模式是一步一步演化发展的，MVVM 是 MVP 的进一步发展与规范，MVP 隔离了 MVC 中的 M 与 V 的直接联系后，靠 Presenter 来中转，所以使用 MVP 时 P 是直接调用 View 的接口来实现对视图的操作的，这个 View 接口的东西一般来说是 showData、showLoading 等。M 与 V 已经隔离了，方便测试，但代码还不够优雅简洁，所以 MVVM 就弥补了这些缺陷。

下面介绍一个 MVVM 架构案例。

VUE.JS 就是基于 MVVM 模式实现出来的一套框架（基于 MVVM 架构的还包括：微信小程序、AngularJS、React）。

VUE 是当前热门的前端框架之一，大部分的网站开发都将 VUE 作为首选。

VUE 中使用了 MVVM 的思想，如图 18-8 所示，其中架构的组件如下：

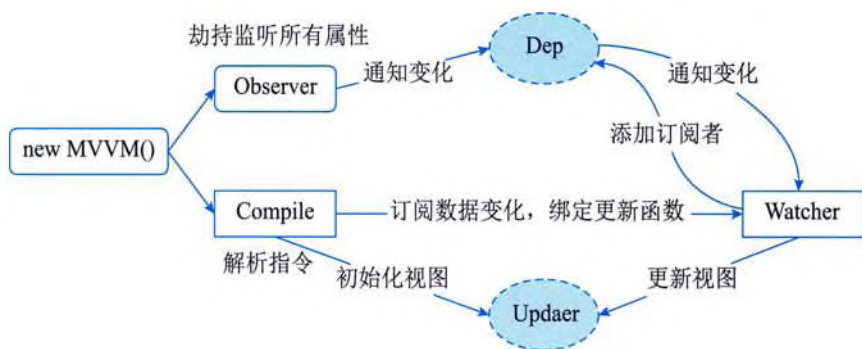


图 18-8 VUE 中的 MVVM 思想

(1) **Observer**。数据监听器，能够对数据对象的所有属性进行监听，如有变动可拿到最新值并通知订阅者，内部采用 SE5 的 `Object.defineProperty` 的 `getter` 和 `setter` 来实现。

(2) **Compile**。指令解析器（`v-for`，`v-if`，...），它的作用是对每个元素节点的指令进行扫描和解析，根据指令模板替换数据，以及绑定相应的更新函数。

(3) **Watcher**。订阅者，作为连接 **Observer** 和 **Compile** 的桥梁，能够订阅并收到每个属性变动的通知，执行指令绑定的相应回调函数。

(4) **Dep**。消息订阅器，内部维护了一个数组，用来收集订阅者（**Watcher**），数据变动触发 `notify` 函数，再调用订阅者的 `update` 方法。

5. 微服务架构阶段

近年来，随着移动应用规模的不断扩大和功能的复杂化，微服务架构逐渐成为了移动应用开发的趋势。该架构将应用程序分解为多个独立的服务，每个服务都可以独立部署和扩展，从而提高了应用的伸缩性和容错性。同时，微服务架构也带来了一些新的挑战，例如，服务间通信和数据一致性问题需要得到更好的解决。

微服务本身的架构也经过很多次的演变，微服务架构演进历程主要经历了以下几个阶段。

1) 基于服务实现模块化

早期的服务化架构主要是将系统拆分为不同的模块并通过服务实现模块化，但服务本质上

仍然是单体应用。

2) 基于 SOA 的服务拆分

随着 SOA 的兴起，微服务架构概念得以深入发展。服务化架构开始通过服务拆分来实现更小的服务颗粒度。同时，服务治理和服务注册中心等基础设施也得以快速发展。

3) 基于容器和服务框架

随着容器技术的成熟，微服务架构开始基于容器进行部署和管理，同时一批开源的服务框架和基础设施也应运而生，如 SpringCloud、Netflix OSS 等。SpringCloud 的微服务示意图如图 18-9 所示。

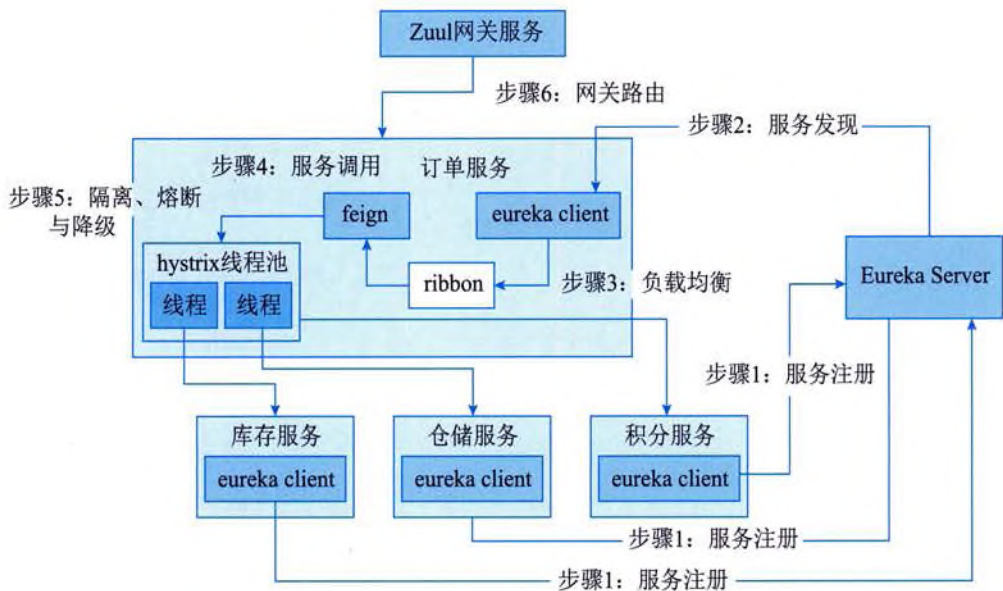


图 18-9 SpringCloud 的微服务示意图

4) 基于云原生的微服务架构

在云原生技术的推动下，微服务架构已经从传统的部署方式转向了基于云平台的统一部署和运维。新的云原生技术如 Kubernetes 等，提供了更高效的容器编排、负载均衡和自动化部署等能力，大大简化了微服务架构的部署和管理。

微服务是一种基于轻量级、面向服务、可独立部署和升级的分布式架构模式，随着企业业务和系统的复杂度不断提升和变化，微服务架构也在不断演进和完善。通过技术的创新、架构思想的转变，微服务架构已经成为各大企业建立基于云原生的分布式系统的首选架构。

6. 微服务架构案例

一个使用了微服务架构的知名案例是 Netflix。Netflix 是一个全球知名的在线流媒体平台，采用了微服务架构来构建其庞大的应用系统。以下是 Netflix 微服务架构的不同功能模块以及它们之间的关系。

(1) 用户管理服务 (User Management Service)。用户管理服务负责处理用户注册、登录、

账号管理等功能。它处理用户身份验证和授权，并维护用户的个人信息。这个服务与其他服务进行通信，以验证用户的访问权限，并确保用户信息的一致性。

(2) 视频流服务 (Video Streaming Service)。视频流服务是 Netflix 的核心服务之一，负责流媒体内容的存储、编码、传输和分发。它处理视频的上传、编码、储存和转码，并通过合适的协议将视频内容传输给用户。这个服务与用户管理服务进行交互，以确保只有经过授权的用户可以访问视频内容。

(3) 推荐系统服务 (Recommendation System Service)。推荐系统服务分析用户的观看历史、评分、喜好等数据，并根据个性化算法生成推荐内容。它与用户管理服务和视频流服务进行通信，获取用户信息和视频内容数据，从而生成针对每个用户的个性化推荐列表。

(4) 支付服务 (Payment Service)。支付服务负责处理用户的订阅和付款操作。它与用户管理服务进行交互，验证用户的付款信息并处理订阅变更和付款事务。支付服务还与其他外部支付网关和服务进行集成，以完成支付操作。

(5) 配置管理服务 (Configuration Management Service)。配置管理服务负责管理整个微服务架构中的配置信息。它存储各个微服务的配置参数，如数据库连接信息、API 密钥等。其他服务可以通过调用配置管理服务来获取其所需的配置信息。

这些不同的功能模块通过轻量级的通信机制（通常是基于 HTTP 的 RESTful API）进行交互。每个微服务运行在自己的进程中，独立部署和扩展。微服务之间通过 API 调用和消息传递进行通信，并通过服务注册和发现机制来实现集中管理。

总的来说，Netflix 采用微服务架构将其庞大的应用系统划分为多个独立的服务，每个服务专注于一个特定的功能领域。这种架构使得 Netflix 能够实现高可用性、可扩展性和灵活性，并且不同功能模块之间的解耦性使得各个模块可以独立开发、部署和扩展。

18.4 移动应用开发

应用开发技术是指使用不同的编程语言和工具来创建和维护各种应用程序的技术。应用开发技术可以分为原生应用开发、混合应用开发和 Web 应用开发等类型，根据不同的平台和需求选择不同的技术方案。

原生应用开发是指针对特定的操作系统（如 iOS 或 Android）使用原生编程语言（如 Objective-C 或 Java）来开发的应用程序，它们可以充分利用设备的硬件和软件功能，提供最佳的用户体验和性能，但是开发难度高，成本高，需要为不同的平台分别开发。

混合应用开发是指使用网页技术（如 HTML5、CSS3、JavaScript）来开发的应用程序，它们可以在不同的平台上运行，具有跨平台的优势，同时也可以通过调用原生 API 来实现一些特定的功能，提高用户体验，但是性能可能不如原生应用，页面交互效果也有限。

Web 应用开发是指使用网页技术来开发的网站或网页，它们不需要下载安装，只需要通过浏览器访问即可，具有便捷性和易维护性的优势，但是用户体验可能不足，页面跳转迟钝或卡顿，功能也受到浏览器的限制。

18.4.1 开发语言

1. 常见开发 App 语言

常见的开发 App 的语言包括：

(1) 原生 App (Native App)。Android 中使用 Java/Kotlin 开发，iOS 中使用 Objective-C/Swift 开发。其优势是效率高；不足是费用高。

(2) Web App。使用 HTML/CSS/JS 编写，运行在浏览器中，封装为 App 安装文件，在应用商店中上架。其优势为一套代码可以运行在两个平台；不足为执行效率不如原生 App。

(3) H5。使用 HTML/CSS/JS 编写，运行在浏览器中，不需要应用商店中上架，也没有图标，通过二维码在朋友圈中快速分享。其优势为一套代码可以运行在两个平台；不足为执行效率不如原生 App。

(4) 混合 / 混编 App (Hybrid App)。使用 HTML/CSS/JS/Native.js(Java/OC=>JS)。其优势为一套代码可以运行在两个平台，还可以调用系统底层原生功能；不足为执行效率不如原生 App。

(5) 桥接 App (Bridged App)。桥接 App 只使用 JS 编写，执行环境会把 JS 编译为 Java/OC，在手机中运行时就是纯原生代码。其优势为一套代码可以运行在两个平台，执行效率高；不足为两个平台不能做到完全一样。

(6) Flutter App。使用 Dart 语言开发，直接在 Android/iOS 手机中的显卡里进行绘图。其优势为一套代码可以运行在两个平台，执行效率高，效果完全一样；不足为全新的语言，Google 开发的进度比较慢。

18.4.2 小程序开发方式

以微信小程序举例，即微信小程序原生开发、混合开发和第三方组件 (SDK) 开发。其中，微信小程序原生开发通常是指使用微信公众号平台提供的开发工具来进行开发，混合开发是指将原生开发与第三方组件开发结合起来，而第三方组件开发则是指使用第三方组件来开发微信小程序。微信小程序的开发技术主要包括 wxml、wxss、js 和 json。其中，wxml 是小程序的模板语言，类似于 HTML；wxss 是小程序的样式语言，类似于 CSS；js 是小程序的逻辑语言，类似于 JavaScript；json 是小程序的配置文件，用于对当前小程序进行全局配置。此外，微信小程序借鉴了很多前端开发的技术理念，它用 React 实现了“视觉组件”，用 CMD 的 require 作为面向对象的 JavaScript，用 Vue 实现了标签式逻辑与数据绑定。

小程序开发方式有多种，主要有以下几种。

1. 基于微信小程序框架开发

使用微信提供的 API 进行小程序开发，新手友好，上手快，可以节省大量开发时间；小程序开发框架的目标是通过尽可能简单、高效的方式让开发者可以在微信中开发具有原生 App 体验的服务。微信团队为小程序提供的框架命名为 MINA。MINA 框架通过封装微信客户端提供的文件系统、网络通信、任务管理、数据安全等基础功能，对上层提供一整套 JavaScript API，让开发者方便地使用微信客户端提供的各种基础功能与能力，快速构建应用。